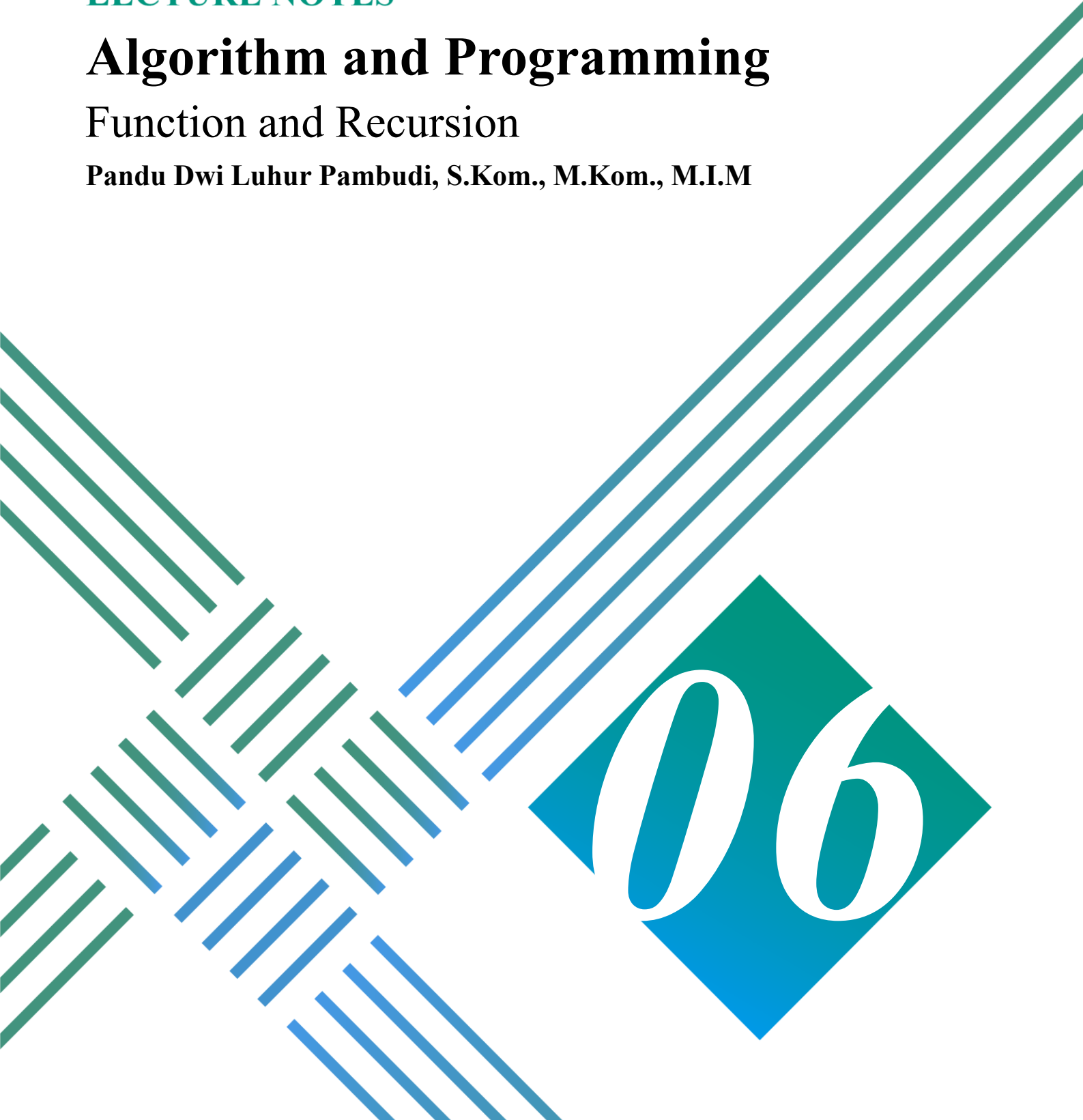


LECTURE NOTES

Algorithm and Programming

Function and Recursion

Pandu Dwi Luhur Pambudi, S.Kom., M.Kom., M.I.M



06

Learning Outcomes

1. LO2: Analyze programs to solve problems using the C language and AI problem-solving sessions related to the topics.

Outline Materi

1. Modular Programming
2. Function
3. Identifier Scoping
4. Passing Parameter
5. Recursion
6. AI Problem-Solving Session: Analyzing Functions and Recursion

6.1

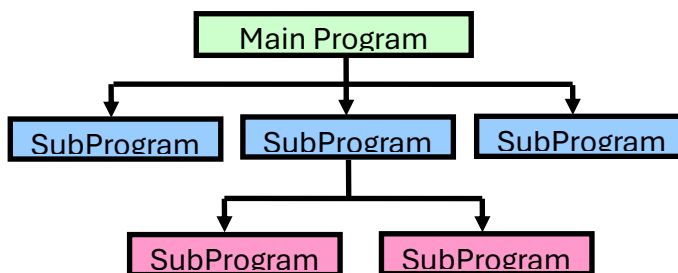
Modular Programming

Dalam pembuatan perancangan program yang baik biasanya digunakan konsep pemrograman modular. Konsep ini merupakan metode untuk memecahkan suatu masalah menjadi bagian-bagian yang lebih kecil (ke dalam modul-modul). Dengan membagi permasalahan tersebut menjadi beberapa modul, maka permasalahan tersebut akan dapat menjadi lebih sederhana, sehingga memudahkan dan lebih mudah memahami dalam penyusunan program. Dalam bahasa C modul dinyatakan dengan menggunakan fungsi (*function*).

Terdapat keuntungan dalam menggunakan modul yaitu :

- Rancangan program besar akan dapat dibagi menjadi modul-modul yang lebih kecil.
- Modul-modul yang telah disederhanakan dapat dikerjakan oleh beberapa orang.
- Mempermudah dalam mencari kesalahan karena alur logika yang telah terstruktur, dan kesalahan dapat ditelusuri dari 1 modul saja.
- Modul ini dapat digunakan berkali-kali jika diperlukan (software reusability).
- Dapat melakukan modifikasi pada suatu modul tanpa mengganggu program secara keseluruhan.
- Mempermudah dokumentasi.

Bahasa pemrograman c mengimplementasikan pemrograman modular menggunakan fungsi. Contoh membagi program menjadi subprogram.



Terdapat sifat-sifat yang menggambarkan modul yang baik :

- Fan-in yang tinggi : makin sering suatu modul dipanggil oleh pemakai, semakin tinggi nilai fan-in.
- Fan-out yang rendah : makin sedikit (spesifik) tugas yang ada pada suatu modul, makin rendah fan-out.
- Self-contained : kemampuan memenuhi kebutuhannya sendiri

Bahan Diskusi



Silahkan diskusikan bersama dosen/tutor/rekan mahasiswa mengenai:

Pertanyaan Diskusi:

Jelaskan bagaimana penerapan konsep modular programming dapat membantu kerja tim dalam mengembangkan program berskala besar. Sertakan contoh pembagian modul di sebuah proyek nyata.

Hint:

- Pikirkan pembagian modul sesuai fitur: input/output, pengolahan data, tampilan, dan validasi.
- Hubungkan dengan pembagian tugas antar anggota tim.
- Bahas manfaat reusability dan parallel development.

Pertanyaan Diskusi:

Berdasarkan sifat-sifat fan-in, fan-out, dan self-contained, diskusikan bagaimana merancang sebuah modul agar memenuhi kriteria modul yang baik. Mengapa penting memperhatikan hal ini?

Hint:

- Fan-in tinggi berarti modul sering dipakai → efisiensi.
- Fan-out rendah artinya tugas modul spesifik → tidak saling bertabrakan.
- Self-contained = modul independen, tidak bergantung berlebihan pada modul lain.
- Pikirkan contoh fungsi hitung rata-rata yang bisa dipanggil di mana saja.

Pertanyaan Diskusi:

Bagaimana modular programming mempermudah proses debugging dan pemeliharaan program di kemudian hari? Jelaskan skenario nyata jika program tidak modular.

Hint:

- Pikirkan alur tracing error: hanya fokus ke satu modul, tidak seluruh program.
- Bandingkan dengan program monolithic: satu error bisa tersebar ke banyak bagian.
- Diskusikan contoh perbaikan bug di fungsi tertentu tanpa risiko merusak fungsi lain.

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini (Modular Programming in C with Hello World Example: [Youtube](#)).

Belajar Mandiri



Pelajari secara mandiri terkait Pengenalan AI pada:

- Hayes, J. R. (2001). Modular programming in C. *Embedded Systems Programming*, 14(13), 18-24.

6.2

Function

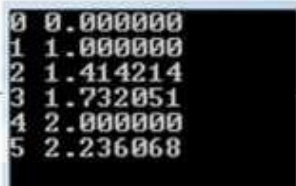
Function merupakan sekelompok pernyataan yang digunakan bersama-sama untuk melakukan tugas. Pada program C memiliki setidaknya 1 fungsi utama, yaitu `main()`; dan pada suatu program dapat membuat fungsi lainnya selain fungsi `main()`;

Function pada bahasa C dibagi menjadi 2 kelompok yaitu :

a. Standart library function

Standar library function adalah fungsi-fungsi yang telah disediakan oleh compiler C, sehingga programmer hanya tinggal menggunakannya saja. Untuk menggunakan standart library function diperlukan header library khusus yang dipanggil untuk mendefinisikan fungsi tersebut. Berikut contoh penggunaan standart library function pada bahasa:

```
#include<stdio.h>
#include<math.h>
int main() {
    int i;
    for(i=0; i<6; i++)
        printf("%d %f \n",i,sqrt(i));
    return 0;
}
```



Pada program tersebut terlihat bahwa terdapat 2 fungsi yang digunakan, yaitu `printf()`, dan `sqrt()`. Kedua fungsi tersebut dapat digunakan jika memanggil header library. Dimana header library dari `printf()` adalah `#include <stdio.h>`, dan header library dari `sqrt()` adalah `#include <math.h>`

b. Programmer-defined function

Programmer-defined function adalah fungsi-fungsi yang dibuat oleh programmer sesuai kebutuhan program. Function yang dibuat dapat digunakan dalam 1 program saja atau dibuat menjadi library yang berisi fungsi-fungsi sejenis.

Dalam merancang function ada beberapa hal yang perlu ditentukan.

- Data yang diperlukan function sebagai masukkan (input) untuk diolah.
- Informasi apa yang dihasilkan (dikembalikan) function kepada pemanggil.
- Algoritma yang digunakan untuk mengolah data masukkan menjadi informasi keluaran.

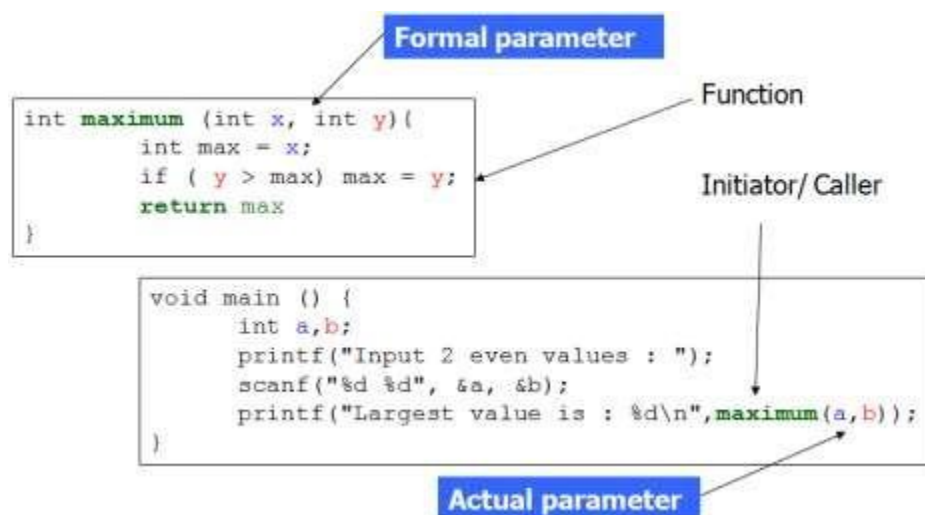
Berikut adalah penggambaran pembuatan function pada bahasa C :

```
return_type function_name( parameter list )
{
    body of the function
}
```

Definisi function pada bahasa C terdiri dari function header, dan function body. Berikut penjelasan dari bagian function :

- *Return type* : return type adalah tipe data dari nilai kembali function. Beberapa function melakukan operasi yang diinginkan tanpa memerlukan pengembalian nilai. Dalam hal ini, void dapat digunakan menjadi return-type yang tidak memerlukan pengembalian nilai.
- *Function name* : penamaan function, digunakan untuk mempermudah pemanggilan function. Penamaan function sama halnya dengan syarat penamaan variabel.
- *Parameters* : sebuah parameter seperti sebuah tempat. Ketika function dipanggil, maka pemanggilan tersebut akan melewati parameter ini. Nilai parameter ini disebut sebagai aktual atau argumen. Banyaknya dan jenis parameter sesuai dengan jumlah parameter yang dipanggil pada function lainnya atau function main.
- *Function body* : function body mengandung kumpulan pernyataan yang mendefinisikan apa yang function tersebut lakukan.

Berikut penggambaran nyata penggunaan function pada bahasa C :



Contoh

```
#include <stdio.h>

int maximum (int x, int y){
    int max = x;
    if ( y > max)      max = y;
    return max
}

void main () {
    int a,b;
    printf("Input 2 even values : ");
    scanf("%d %d", &a, &b);
    printf("Largest value : %d\n",maximum(a,b));
}
```

No need for
function
prototype as the
function placed
above initiator

Ada beberapa jenis pengidentifikasi dengan durasi penyimpanan statis: pengidentifikasi eksternal (seperti variabel global dan nama fungsi) dan variabel lokal yang dideklarasikan dengan specifier kelas penyimpanan statis. Variabel global dan nama fungsi memiliki kelas penyimpanan di luar secara default. Variabel global dibuat dengan menempatkan deklarasi variabel di luar definisi fungsi apa pun. Mereka mempertahankan nilai-nilai mereka sepanjang eksekusi program.

Variabel dan fungsi global dapat dirujuk oleh fungsi apa pun yang mengikuti deklarasi atau definisi mereka dalam file. Ini adalah salah satu alasan untuk menggunakan prototipe fungsi - ketika kami menyertakan `stdio.h` dalam program yang memanggil `printf`, prototipe fungsi ditempatkan pada awal file kami untuk membuat nama `printf` diketahui oleh file lainnya.

Mendefinisikan variabel sebagai global daripada lokal memungkinkan efek samping yang tidak diinginkan terjadi ketika fungsi yang tidak memerlukan akses ke variabel secara tidak sengaja atau maliciously memodifikasinya. Secara umum, Anda harus menghindari variabel

global kecuali dalam situasi dengan persyaratan kinerja yang unik. Variabel yang hanya digunakan dalam fungsi tertentu harus didefinisikan sebagai variabel lokal dalam fungsi itu.

Variabel statis lokal masih diketahui hanya dalam fungsi di mana mereka didefinisikan dan mempertahankan nilainya ketika fungsi kembali. selanjutnya fungsi dipanggil, variabel lokal statis berisi nilai yang dimilikinya ketika fungsi terakhir keluar.

Bahan Diskusi



Silahkan diskusikan bersama dosen/tutor/rekan mahasiswa mengenai:

Pertanyaan Diskusi:

Apa perbedaan mendasar antara *standar library function* dan *programmer-defined function* dalam bahasa C? Bagaimana cara memilih kapan menggunakan fungsi pustaka standar dan kapan harus membuat fungsi sendiri?

Hint:

- Diskusikan contoh nyata fungsi pustaka seperti `printf()` atau `sqrt()` dibandingkan fungsi buatan seperti `maximum()`.
- Bahas kelebihan menggunakan fungsi pustaka (praktis, sudah teruji) vs fungsi buatan (fleksibel, sesuai kebutuhan spesifik program).

Pertanyaan Diskusi:

Mengapa penting mendefinisikan *parameter list* dan *return type* dengan benar saat membuat fungsi? Diskusikan apa risiko jika terjadi ketidaksesuaian tipe data parameter atau return value saat fungsi dipanggil.

Hint:

- Bicarakan contoh: Apa yang terjadi jika sebuah fungsi diharapkan mengembalikan `int` tapi tidak ada `return`?
- Diskusikan pengaruh type mismatch pada hasil output dan potensi error saat kompilasi.

Pertanyaan Diskusi:

Kapan sebaiknya menggunakan variabel global dan kapan sebaiknya menggunakan variabel lokal di dalam fungsi? Apa risiko yang muncul jika variabel global digunakan secara tidak hati-hati?

Hint:

- Bahas contoh kasus: variabel global yang diubah secara tak sengaja oleh fungsi lain.
- Diskusikan bagaimana variabel lokal membuat fungsi lebih aman, lebih modular, dan mudah diuji.
- Tambahkan bahasan tentang *static local variable* dan bagaimana perilakunya berbeda dengan variabel lokal biasa.

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini (C Functions: [Youtube](#)).

Belajar Mandiri



Pelajari secara mandiri terkait Pengenalan AI pada:

- Buonanno, E. (2022). *Functional Programming in C*. Simon and Schuster.

6.3 Identifier Scoping

Berdasarkan jangkauannya identifier dibagi menjadi 2, yaitu :

a. Global identifier

Global identifier merupakan identifier yang dideklarasikan di luar function dan ditempatkan di atas semua function dalam suatu program. Jangkauannya biasanya meliputi seluruh program. Identifier yang dideklarasikan secara global, dapat dideklarasikan kembali di dalam function.

b. Local identifier

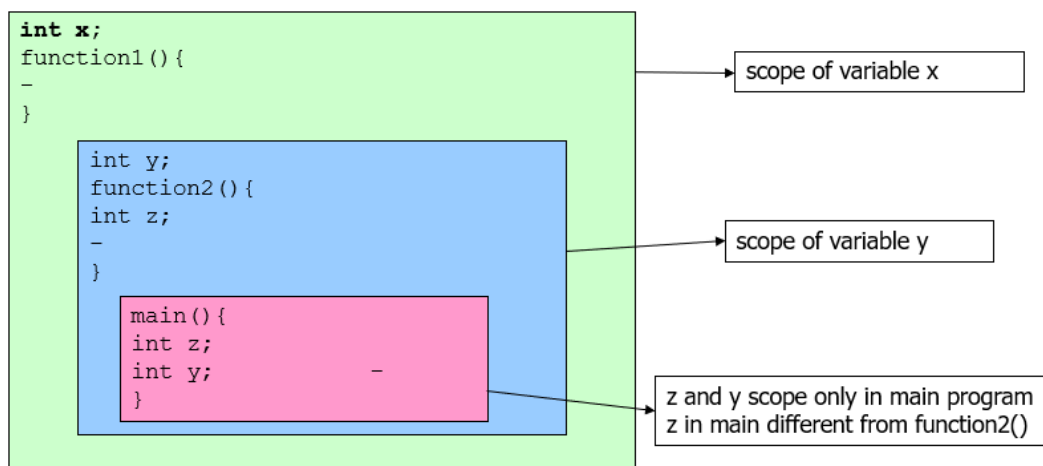
Local identifier merupakan identifier yang dideklarasikan di dalam function dan ditempatkan di atas semua function dalam suatu program. Jangkauannya biasanya meliputi seluruh program. Identifier yang dideklarasikan secara global, dapat dideklarasikan kembali di dalam function.

Keuntungan :

- Transfer data antar function menjadi mudah.

Kerugian :

- Jika program semakin besar, maka semakin besar juga kecenderungan terjadinya kesalahan.
- Sulit melacak kesalahan.
- Data tidak terjaga dengan baik, setiap function dapat mengubah isi variabel tanpa sepengetahuan function lainnya.



Bahan Diskusi



Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

Pertanyaan Diskusi:

Apa perbedaan mendasar antara *global identifier* dan *local identifier* dalam bahasa C? Berikan contoh praktis bagaimana keduanya digunakan dalam sebuah program.

Hint:

- Diskusikan penempatan deklarasi: global di luar semua fungsi, lokal di dalam fungsi.
- Contoh: variabel `int count;` di luar fungsi `main()` vs `int count;` di dalam fungsi `main()`.

Pertanyaan Diskusi:

Mengapa programmer terkadang memilih menggunakan *global identifier*? Dalam situasi apa penggunaan identifier global lebih menguntungkan dibandingkan variabel lokal?

Hint:

- Bahas keuntungan: mempermudah transfer data antar fungsi tanpa harus menggunakan parameter.
- Contoh kasus: variabel status yang harus diakses oleh banyak fungsi (misalnya `statusLogin`).

Pertanyaan Diskusi:

Apa saja risiko dan tantangan yang muncul jika terlalu sering menggunakan *global identifier* dalam program berskala besar? Bagaimana cara meminimalkan risiko tersebut?

Hint:

- Diskusikan potensi bug: variabel global dapat diubah fungsi lain secara tak terduga.
- Bahas bagaimana debugging menjadi sulit.
- Solusi: utamakan variabel lokal dan gunakan parameter untuk komunikasi antar fungsi, batasi variabel global hanya untuk kebutuhan khusus.

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini (C Variable Scope: [Youtube](#)).

6.4 Passing Parameter

Jika satu function memanggil function yang lain untuk melakukan tugas tertentu, kepada function yang dipanggil (called function) diberikan seluruh data yang diperlukan untuk melakukan tugas yang diberikan oleh function pemanggil. Setelah melakukan tugas function yang dipanggil mengembalikan hasil proses kepada function pemanggil (calling function). Pengiriman data tersebut melalui parameter yang menjadi interface (penghubung) suatu modul dengan modul yang lain. Pengiriman parameter di dalam bahasa C terdapat 2 jenis yaitu :

- Call by value

Call by value, adalah pengiriman ke function lain berupa nilai (r-value). Nilai parameter aktual akan disalin ke parameter formal. Dengan cara ini nilai parameter aktual tidak berubah sekalipun terjadi perubahan nilai parameter formal. Berikut contoh program function call by value pada bahasa C :

```
#include<stdio.h>

void tukar(int x, int y)    // formal parameter
{
    int z; //variabel sementara
    z = x;
    x = y;
    y = z;
    printf("Hasil nilai yang ditukar\n");
    printf("a = %d b = %d\n", x, y);
}

int main()
{
    int a = 100, b = 200;

    printf("Nilai sebelum ditukar\n");
    printf("a = %d b = %d\n", a, b);
    tukar(a, b);        // actual parameter

    getchar();
    return 0;
}
```

Pada program tersebut dapat dilihat pada saat di function main memanggil fungsi tukar untuk melakukan tugas penukaran variabel a dan b. Maka fungsi tukar akan melakukan tugasnya dan mengembalikan hasil proses yang ada dalam fungsi tukar tersebut.

Berikut hasil output dari program function call by value :

```
Nilai sebelum ditukar
a = 100 b = 200
Hasil nilai yang ditukar
a = 200 b = 100
```

- Call by reference

Pemanggilan dengan alamat yang dilakukan dengan cara mengirimkan alamat-alamat variabel melalui parameter aktualnya ke function yang dipanggil. Function yang dipanggil kemudian menampung alamat-alamat ini pada variabel-variabel pointer yang terdapat dalam daftar parameter formal. Dengan cara pemanggilan ini, jika terdapat perubahan terhadap data yang dikirimkan, perubahan yang terjadi tidak hanya secara lokal saja. Semua perubahan data yang dilakukan pada function yang dipanggil akan otomatis mengubah data pada function yang memanggil. Hal ini disebabkan informasi yang dikirimkan ke function yang dipanggil adalah alamat data tersebut, sehingga perubahan terhadap data dilakukan langsung di tempat data itu tersimpan. Call by reference disebut juga call by location.

Berikut adalah contoh dari function call by reference pada bahasa C :

```
#include<stdio.h>

void tukar(int *x, int *y)    // formal parameter
{
    int z; //variabel sementara
    z = *x;
    *x = *y;
    *y = z;
}

int main()
{
    int a = 100, b = 200;

    printf("Nilai sebelum ditukar\n");
    printf("a = %d b = %d\n", a, b);
    tukar(&a, &b);           // actual parameter
    printf("Hasil nilai yang ditukar\n");
    printf("a = %d b = %d\n", a, b);

    getchar();
    return 0;
}
```

Pada program ini hasil nilai yang telah ditukar tidak dapat dicetak langsung pada fungsi tukar. Hal ini dikarenakan jika mencetak langsung dari fungsi tukar maka yang akan dicetak adalah alamat dari nilai x, dan y. Karena nilai x dan y berasal dari alamat &a, dan &b.

Program tersebut akan menghasilkan output yang sama dengan function call by value.

```
Nilai sebelum ditukar
a = 100 b = 200
Hasil nilai yang ditukar
a = 200 b = 100
```

Bahan Diskusi



Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

Pertanyaan Diskusi:

Jelaskan perbedaan mendasar antara *call by value* dan *call by reference* dalam bahasa C. Berikan contoh situasi praktis di mana masing-masing metode lebih cocok digunakan.

Hint:

- *Call by value* hanya mengirim salinan nilai ke fungsi, data aslinya tetap aman.
- *Call by reference* mengirimkan alamat memori, sehingga fungsi dapat mengubah nilai aslinya.
- Contoh praktis: hitung luas (pakai value), tukar nilai variabel (pakai reference).

Pertanyaan Diskusi:

Apa risiko yang perlu diperhatikan saat menggunakan *call by reference*? Bagaimana programmer dapat menghindari terjadinya perubahan nilai yang tidak diinginkan?

Hint:

- Diskusikan efek samping: nilai asli bisa berubah tanpa sengaja.
- Programmer dapat menggunakan pointer secara hati-hati dan mendokumentasikan fungsi dengan jelas.
- Pertimbangkan kapan sebaiknya tetap memakai *call by value* demi keamanan data.

Pertanyaan Diskusi:

Bagaimana hasil output akan berbeda antara program yang menggunakan *call by value* dengan *call by reference* jika fungsi melakukan perubahan pada parameter?

Hint:

- *Call by value*: perubahan hanya terjadi pada salinan di dalam fungsi, nilai asli di fungsi pemanggil tetap.
- *Call by reference*: perubahan di fungsi berdampak langsung ke variabel di fungsi pemanggil.
- Analisis output dengan membandingkan variabel sebelum dan sesudah fungsi dipanggil.

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini (C Variable Scope: [Youtube](#)).

6.5

Recursion

Rekursif (recursion) adalah suatu proses yang memanggil dirinya kembali. Dalam bahasa C rekursif dilakukan oleh function. Function yang memanggil dirinya sendiri disebut sebagai recursive function. Berikut adalah bentuk sintaks fungsi rekursif pada bahasa C :

```
void recursion()
{
    recursion();      /* function calls itself */
}

int main()
{
    recursion();
}
```

Berikut contoh program rekursif pada bahasa C :

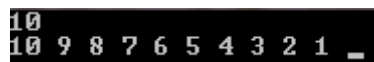
```
#include<stdio.h>

void count ( int x )
{
    if ( x > 0 )
    {
        printf("%d ",x);
        count(x-1);
    }
}

int main()
{
    int bil;
    scanf("%d", &bil);
    count(bil);

    getchar();
    return 0;
}
```

Pada awal program tersebut diminta untuk memasukkan inputan, jika user memasukkan 10. Maka bilangan 10 akan disimpan pada variabel bil. Pada fungsi main dipanggil fungsi count, yang mana didalam struktur fungsi count, terdapat rekursi yang dijalankan untuk mencetak 10 hingga 0 (batasan yang telah ditentukan pada program) dari program tersebut akan menghasilkan output sebagai berikut:



```
10
10 9 8 7 6 5 4 3 2 1 _
```

Dalam penggunaanya rekursif memiliki kekurangan yaitu:

- Memerlukan memori yang lebih banyak untuk menyimpan activation record dan variabel. Activation record diperlukan waktu proses kembali kepada fungsi pemanggil.
- Memerlukan waktu yang lebih lama untuk menangani activation record.

Secara umum penggunaan rekursif dilakukan hanya jika :

- Penyelesaian tidak dapat dilakukan secara iteratif (perulangan).
- Efisiensi dengan cara rekursif sudah sesuai.
- Fungsi rekursif setidaknya memiliki 1 kondisi yang digunakan untuk keluar dari proses rekursif tersebut, jika tidak maka fungsi rekursif akan memanggil dirinya sendiri sampai terjadi runtime stack overflows.

Perbedaan Iterative vs. Recursive jika sebuah masalah dapat diselesaikan dengan cara iteratif maka gunakan iteratif. Jika penyelesaian dengan iteratif memerlukan algoritma yang cukup rumit maka dapat menggunakan pendekatan rekursif karena rekursif memerlukan sumber daya yang banyak.

6.6

AI Problem-Solving Session: Analyzing Functions and Recursion

HAICOLA Section: Mahasiswa dapat menggunakan AI Code Assistant (seperti **Gemini** atau **ChatGPT**) untuk melakukan pengecekan sintaks, memahami error, dan memperbaiki kode C secara efisien.

Fungsi dan, khususnya, **Rekursi** adalah konsep penting yang mengubah pemikiran pemrograman Anda dari sekuensial menjadi modular dan iteratif secara mandiri. Kesalahan dalam area ini sering kali melibatkan kesulitan dalam melacak aliran eksekusi dan pengelolaan memori (*stack*).

Fokus: Analisis Rekursi dan *Parameter Passing*

Kemampuan untuk secara akurat melacak panggilan fungsi rekursif dan memahami bagaimana parameter diteruskan sangat penting untuk **LO 2**. Tantangan utama meliputi:

1. **Logika Fungsi Rekursif:** Memastikan bahwa **basis kasus** (*base case*) telah didefinisikan dengan benar untuk menghentikan rekursi, dan **langkah rekursif** (*recursive step*) bergerak menuju kasus dasar. Kegagalan di sini dapat menyebabkan *Infinite Recursion* dan **Stack Overflow**
2. **Kesalahan Parameter Passing:** Memahami perbedaan antara Pass-by-Value (nilai disalin) dan Pass-by-Reference (menggunakan pointers), yang sangat menentukan apakah perubahan di dalam fungsi memengaruhi variabel asli di luar fungsi.

Contoh Praktis: Menggunakan AI untuk Analisis Rekursi

Anda dapat menggunakan AI Code Assistant (seperti Gemini) untuk meminta alat tersebut **melacak** (*trace*) eksekusi fungsi rekursif Anda. Ini akan membantu Anda memvisualisasikan *call stack* tanpa harus melakukannya secara manual.

Skenario 1: Rekursi yang Bermasalah (Basis Kasus Hilang)

Kode yang Bermasalah - Menghitung Faktorial:

```
#include <stdio.h>

long factorial(int n) {
    // Basis kasus hilang atau salah!
    return n * factorial(n - 1);
}

int main() {
    printf("Factorial 3: %ld\n", factorial(3));
    return 0;
}
```

Permintaan (Prompt) ke AI:

"Kode C saya di bawah ini mencoba menghitung faktorial tetapi gagal atau menghasilkan output yang aneh. Tolong jelaskan mengapa ini terjadi dan bagaimana rekursi harusnya dihentikan."

Respon AI yang Diharapkan:

AI akan menjelaskan bahwa kode tersebut kekurangan **basis kasus** yang menentukan kapan rekursi harus berhenti (seharusnya ketika $n = 0$ atau $n = 1$). Karena tidak ada basis kasus, fungsi akan terus memanggil dirinya sendiri dengan nilai n yang semakin negatif, menyebabkan **Stack Overflow**.

Skenario 2: Menganalisis *Pass-by-Value* vs. *Pass-by-Reference*

Kode Analisis - Swapping Variables:

```
#include <stdio.h>

void swap_by_value(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
} // Variabel asli di main() tidak akan berubah

void swap_by_reference(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
} // Variabel asli di main() akan berubah

int main() {
    int x = 10, y = 20;
    // swap_by_value(x, y); // Coba ini dulu
    swap_by_reference(&x, &y); // Coba ini setelahnya
    printf("X: %d, Y: %d\n", x, y);
    return 0;
}
```

Pemintaan (Prompt) ke AI:

"Saya punya dua fungsi swap di C: satu menggunakan nilai (pass-by-value) dan satu menggunakan pointer (pass-by-reference). Jelaskan perbedaan utama dalam hal bagaimana variabel 'x' dan 'y' di fungsi main() dipengaruhi oleh masing-masing fungsi swap."

Respon AI yang Diharapkan:

AI akan menjelaskan bahwa swap_by_value hanya mengubah salinan lokal dari x dan y, sehingga nilai asli tetap 10 dan 20. Sementara itu, swap_by_reference menggunakan pointer untuk mengakses alamat memori variabel asli, sehingga nilai x dan y berhasil dipertukarkan menjadi 20 dan 10 di fungsi main().

Praktik Terbaik untuk Mahasiswa

- **Pikirkan *Stack*:** Saat menghadapi rekursi, gunakan AI untuk "melihat" aliran eksekusi (*trace*). Pikirkan setiap panggilan rekursif sebagai lapisan baru di atas *call stack*.
- **Perjelas *Pointers*:** Saat *debugging parameter passing*, gunakan AI untuk memverifikasi apakah *pointer* digunakan dengan benar (misalnya, kapan menggunakan `&` dan kapan menggunakan `*`). Ini sangat penting untuk memecahkan masalah (LO 2).
- **Latihan *Employability Skill*:** Menganalisis kode fungsi yang kompleks dan memperbaiki kesalahan logika rekursi adalah latihan langsung dalam **Problem-Solving and Decision-Making**.

Bahan Diskusi



Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

Pertanyaan Diskusi:

Jelaskan bagaimana cara kerja rekursif dalam bahasa C dan apa peran kondisi basis (*base case*) pada sebuah fungsi rekursif.

Hint:

- Rekursif berarti fungsi memanggil dirinya sendiri.
- Tanpa *base case*, rekursif akan berjalan terus menerus hingga stack overflow.
- *Base case* adalah kondisi untuk berhenti memanggil dirinya sendiri.

Pertanyaan Diskusi:

Bandingkan kelebihan dan kekurangan penggunaan rekursi dibandingkan iterasi (*looping*) pada penyelesaian suatu masalah. Berikan contoh masalah yang lebih efisien diselesaikan dengan rekursi.

Hint:

- Iterasi biasanya lebih hemat memori, rekursi lebih sederhana untuk masalah yang bercabang.
- Contoh: pohon biner, faktorial, *Tower of Hanoi*.
- Diskusikan aspek keterbacaan kode dan konsumsi sumber daya.

Pertanyaan Diskusi:

Jika sebuah fungsi rekursif tidak memiliki *base case*, apa konsekuensinya pada program? Bagaimana cara menghindari error seperti *stack overflow*?

Hint:

- Tanpa *base case*, fungsi akan terus memanggil diri sendiri → memori habis → error.
- Selalu rancang kondisi berhenti yang pasti tercapai.
- Lakukan pengujian input agar rekursif tidak menimbulkan error runtime.

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini (Recursion in C: [Youtube](#)).

Post-Test

Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

Soal 1 — Modular Programming

Manakah pernyataan berikut yang BUKAN keuntungan dari modular programming?

- A. Mempermudah dokumentasi
- B. Modul dapat digunakan kembali (reusability)
- C. Semua data harus disimpan di variabel global
- D. Memudahkan pembagian kerja dalam tim

Soal 2 — Function

Dalam bahasa C, fungsi `sqrt()` dapat digunakan hanya jika program Anda menyertakan...

- A. `#include <stdio.h>`
- B. `#include <math.h>`
- C. `#include <stdlib.h>`
- D. `#include <conio.h>`

Soal 3 — Identifier Scoping

Manakah pernyataan yang benar tentang variabel global dan lokal di C?

- A. Variabel global hanya bisa diakses di dalam fungsi yang mendeklarasikannya
- B. Variabel lokal bisa diakses di mana saja dalam program
- C. Variabel global dapat diakses oleh semua fungsi setelah deklarasi
- D. Variabel lokal dapat dipanggil di fungsi lain secara otomatis

Soal 4 — Passing Parameter

Perbedaan utama antara Call by Value dan Call by Reference dalam C adalah...

- A. Call by Value mengirim alamat memori, Call by Reference mengirim salinan nilai
- B. Call by Value tidak mempengaruhi nilai asli, Call by Reference dapat mengubah nilai asli
- C. Keduanya selalu mengubah nilai variabel di main
- D. Call by Reference hanya dapat digunakan dengan integer

Soal 5 — Recursion

Apa risiko utama jika fungsi rekursif tidak memiliki kondisi berhenti (*base case*)?

- A. Program akan menghasilkan output yang salah tetapi tetap berjalan
- B. Program akan langsung berhenti tanpa error
- C. Program akan terus memanggil dirinya sendiri hingga stack overflow
- D. Program akan diubah otomatis menjadi iterasi

Kesimpulan

1. Dalam pembuatan perancangan program yang baik biasanya digunakan konsep pemrograman modular. Konsep ini merupakan metode untuk memecahkan suatu masalah menjadi bagian-bagian yang lebih kecil (ke dalam modul-modul). Function merupakan sekelompok pernyataan yang digunakan bersama-sama untuk melakukan tugas. Pada program C memiliki setidaknya 1 fungsi utama, yaitu `main()`; dan pada suatu program dapat membuat fungsi lainnya selain fungsi `main()`;
2. Function pada bahasa C dibagi menjadi 2 kelompok yaitu: Standard library function. dan Programmer-defined function
3. Identifier Scoping, berdasarkan jangkauannya identifier dibagi menjadi 2, yaitu: Global identifier merupakan identifier yang dideklarasikan di luar function dan ditempatkan di atas semua function dalam suatu program dan Local identifier merupakan identifier yang dideklarasikan di dalam function, termasuk dalam daftar parameter. Jangkauannya terbatas pada function itu sendiri.
4. Passing Parameter merupakan suatu function yang memanggil function yang lain untuk melakukan tugas tertentu, kepada function yang dipanggil (called function) diberikan seluruh data yang diperlukan untuk melakukan tugas yang diberikan oleh function pemanggil. Pengiriman parameter di dalam bahasa C terdapat 2 jenis yaitu Call by value dan Call by reference / location
5. Rekursif (recursion) adalah suatu proses yang memanggil dirinya kembali. Dalam bahasa C rekursif dilakukan oleh function.

Daftar Pustaka

- Paul Deitel & Harvey Deitel. (2022). C how to program. 09. Pearson Education. Hoboken. ISBN: 978-0-13-739839-3. Chapter 5
- Functions in C: <http://aelinik.free.fr/c/ch15.htm>
- Hayes, J. R. (2001). Modular programming in C. *Embedded Systems Programming*, 14(13), 18-24.
- Buonanno, E. (2022). *Functional Programming in C*. Simon and Schuster.

Berikut jawaban untuk pertanyaan Post-Test diatas:

1. **Jawaban:** C

Alasan: Modular programming justru mendorong penggunaan variabel lokal untuk menjaga independensi modul.

2. **Jawaban:** B

Alasan: `sqrt()` adalah fungsi library standar untuk akar kuadrat yang disediakan oleh header `<math.h>`.

3. **Jawaban:** C

Alasan: Variabel global memiliki cakupan seluruh file/program setelah deklarasi.

4. **Jawaban:** B

Alasan: Call by Value hanya mengirim salinan, sedangkan Call by Reference memungkinkan perubahan langsung pada data aslinya.

5. **Jawaban:** C

Alasan: Tanpa *base case*, rekursi berjalan tanpa henti dan akan menyebabkan error *runtime stack overflow*.